

Welcome ÷

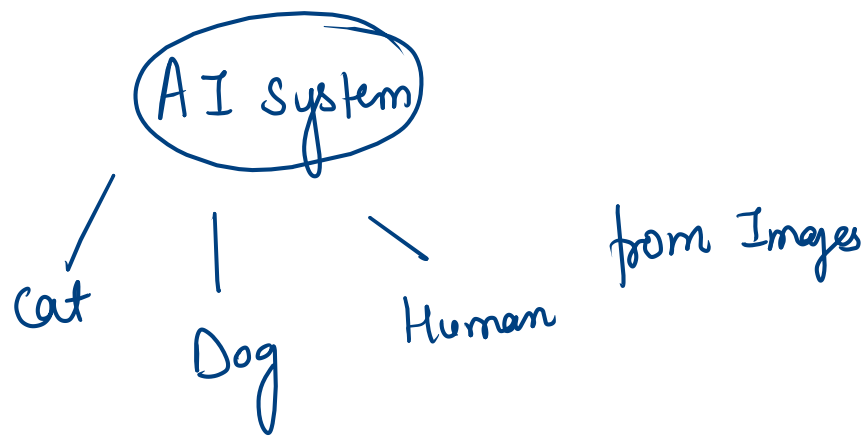
Agenda



CNN

detail

ANN



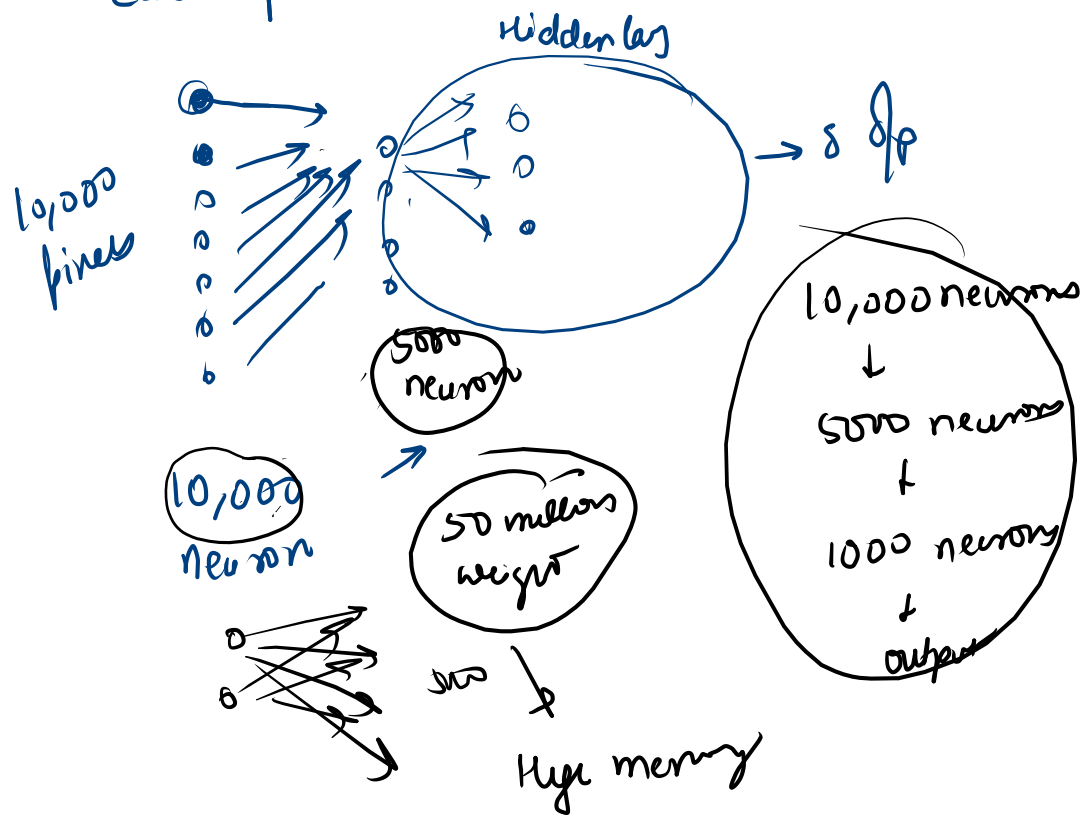
Without CNN ÷



100 x 100 pixels.
 Total pixels = 100 x 100
 = 10,000 pixels

R
G
B

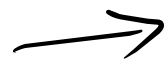
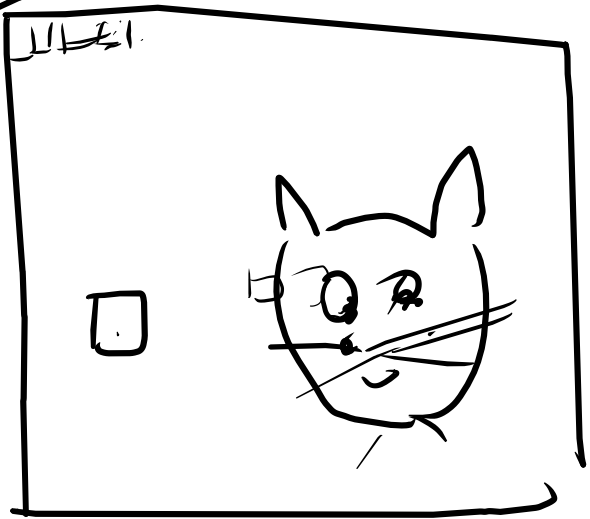
Each pixel becomes an input neurons



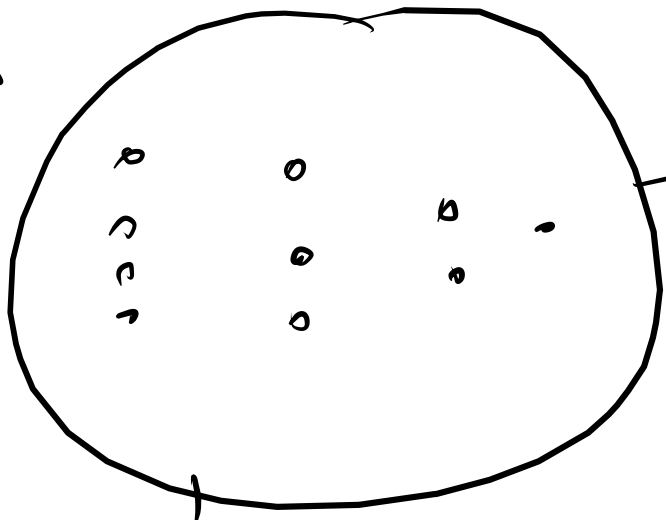
$$\begin{aligned}
 &w_1x_1 + w_2x_2 + w_3x_3 \\
 &w_1'x_1 + w_2'x_2 + w_3'x_3
 \end{aligned}$$

Memory unsupport
 Huge training time.

Problem 2



o
o
o
o
o
o
o
o
o
o



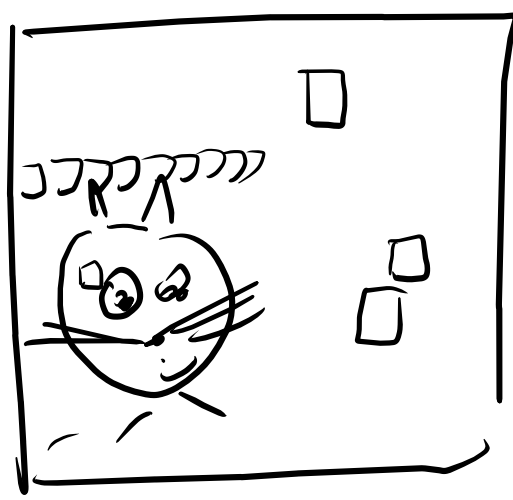
Hidden layer

output
=

Cat

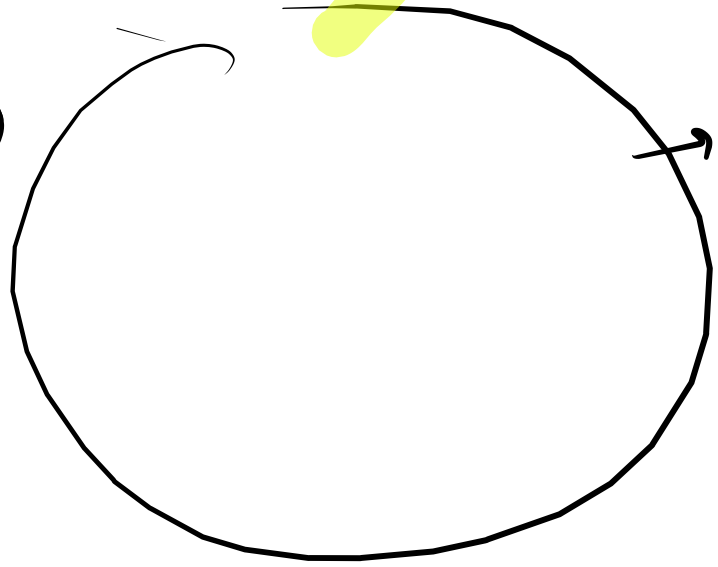


Rotate same imp



o
o
o
o
o
o
o
o
o
o

Input



output

CNN architecture

Image

convolution layer

ReLU

(activation fun)

pooling

convolution layer

pooling

flatten

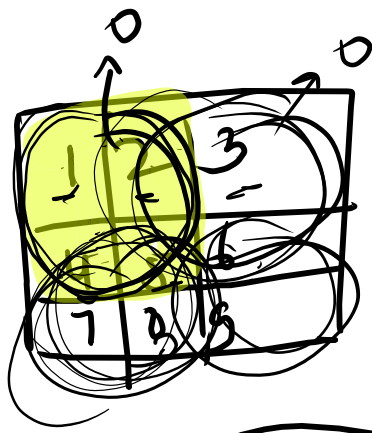
Dense layer
(output) σ

CNN $\rightarrow$ convolution W/O
mimics how humans look at images.

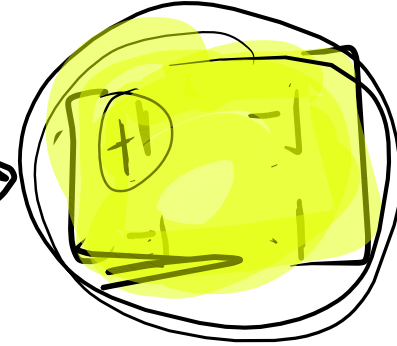
max(1,0)

Easy way

Step 1 → Suppose Image



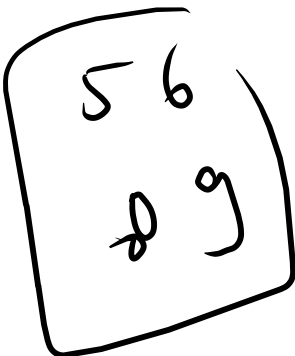
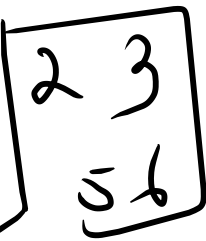
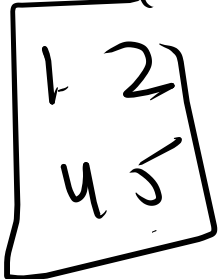
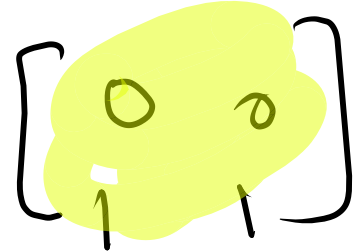
magnify which
look at
2x2 part of image



RGB

← diagonal

(Trained the model on millions of Images)



filter learns ÷

$$1 \times 1 - 1 \times 2 - 1 \times 4 + 5 \times 1$$

$$= 1 - 2 - 4 + 5$$

$$= 0$$

Vertical edges (filter 1)

Horizontal edges (filter 2)

Curves (filter 3)

eyes (filter 4)

()

$$2 \times 1 + 3 \times (-1) +$$

$$5 \times (-1) + 6 \times 1 = 2 - 3 - 5 + 6 = -8 + 8 = 0$$

Step 2

ReLU

filter $\rightarrow$ diagonal $\rightarrow -5$

curve $\rightarrow 2$

circle $\rightarrow 10$

H edge $\rightarrow 20$

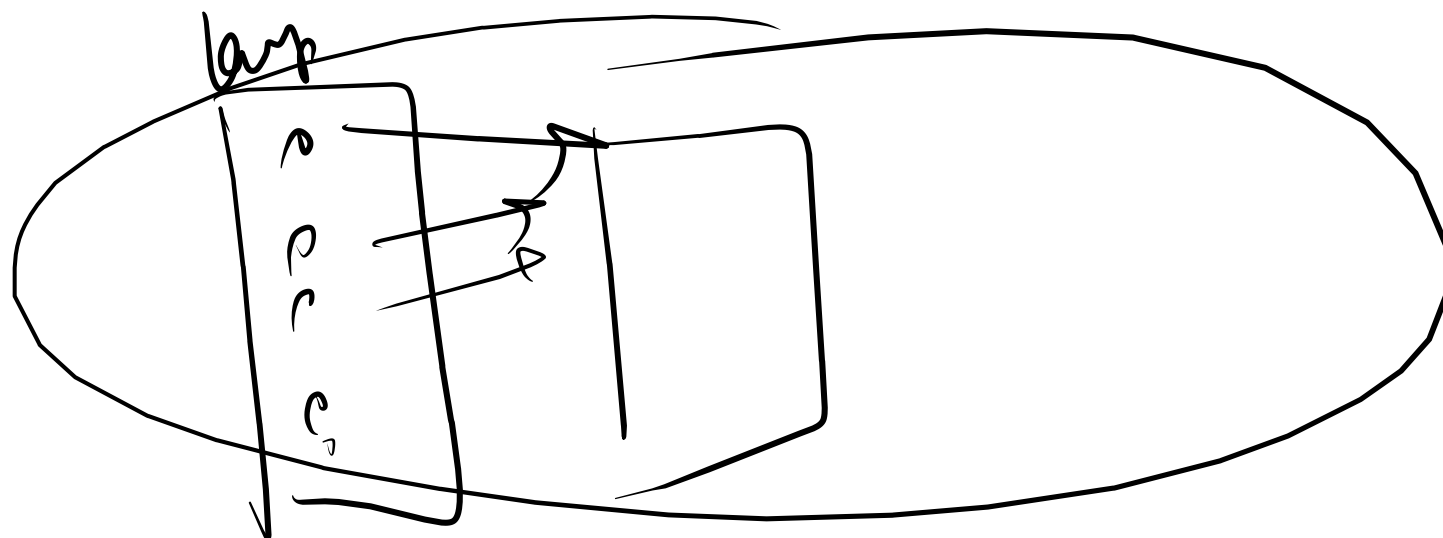
$\max(x, 0)$

$\rightarrow 0 \rightarrow$ do not activate the neuron.

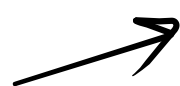
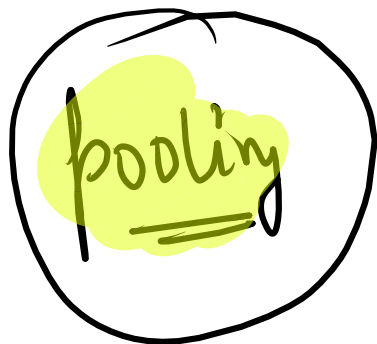
$\rightarrow 2$

$\rightarrow 10$

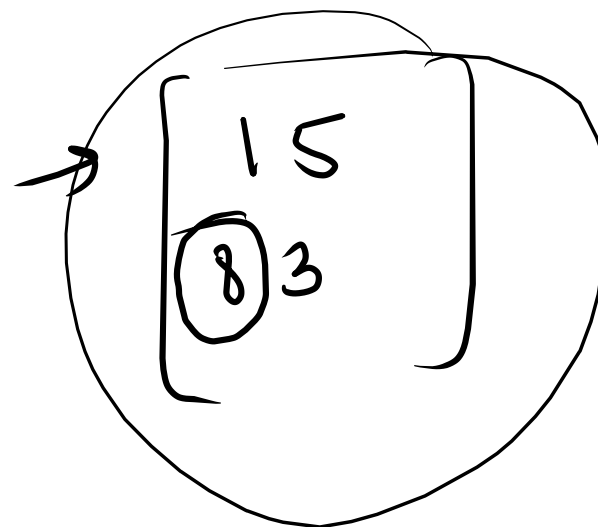
$\rightarrow 20$



step 3



Suppose output



Keep the most
important information
& throw away unnecessary
details

Max pooling: ?

Largest value survives.

Large feature
maps

High
computation

Overfitting

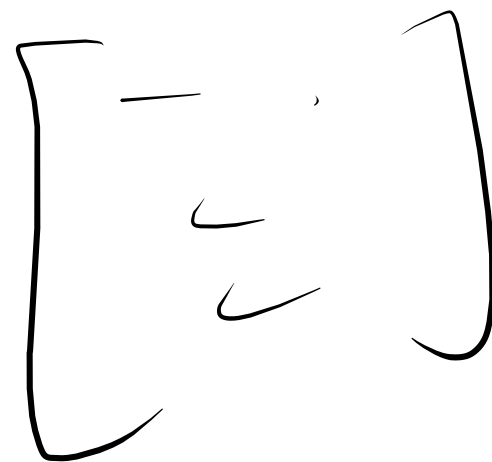
Type of pooling

max pooling

Average pooling

Instead of max
Take average.

Global pooling



Average the
entire set.

Step 4 → Multiple convolution layers.

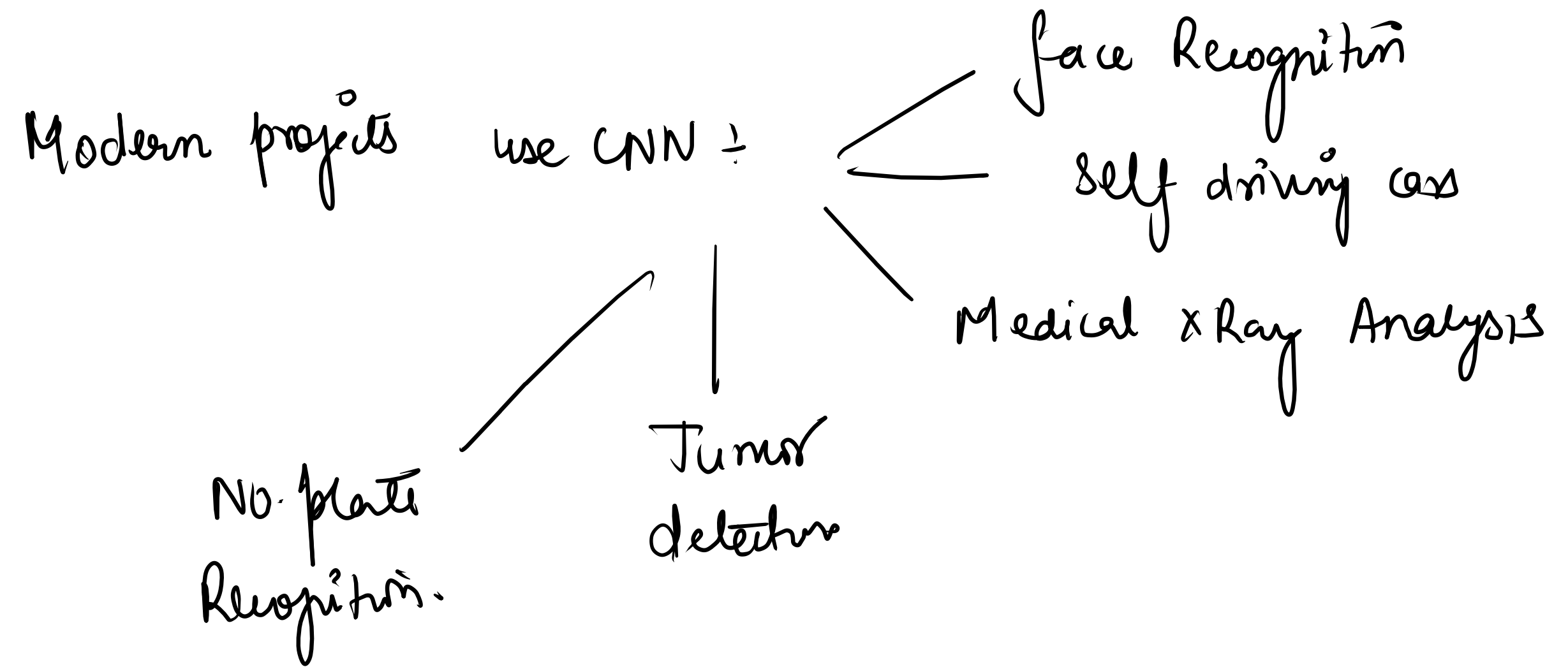
first layer → edges
lines
corners } filters identify it

second layer learns → eyes
nose }

third layer → Cat face ✓
Dog face ✓

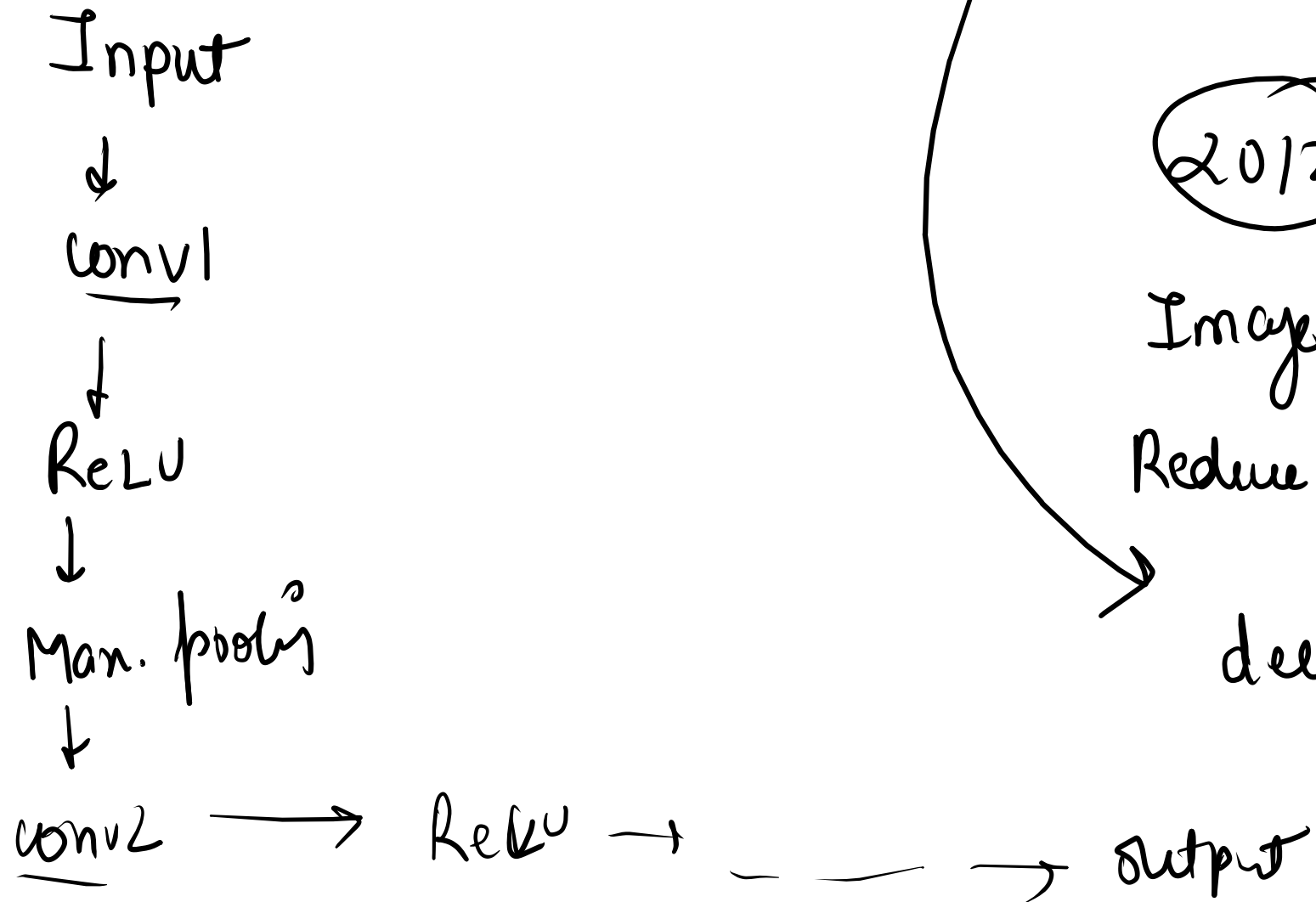
Dense layer
Fourth layer
→ complete animal

Kat = 95%
Dog = 3%
Rabbit = 2%



Two major CNN architecture

AlexNet architecture



AlexNet (2012)

Before 2012, image recognition accuracy was very poor.

(2012) → AlexNet

ImageNet competition

Reduce error rate drastically

deep learning revolution came

AlexNet Key Innovations

① ReLU Activation

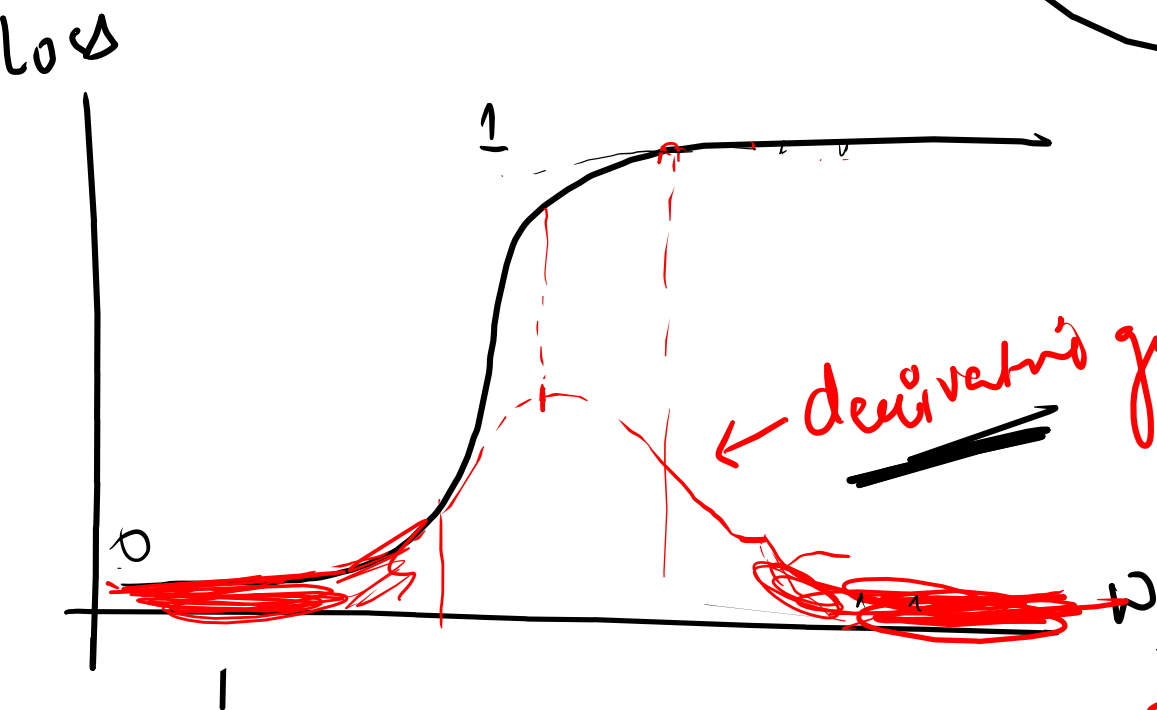
Before
Alex net

Sigmoid
Tanh

$$f(x) = \max(0, x)$$

★ Faster training

↖ less vanishing gradient problem



$\frac{dL}{dw} =$ what will be
change in loss
when you increase
weights by

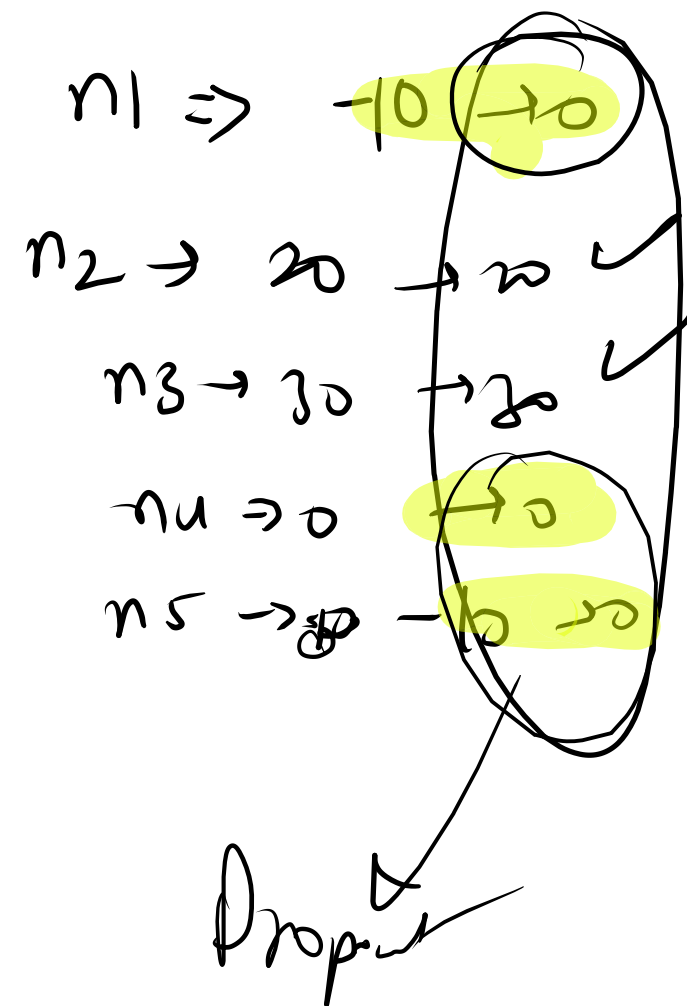
② Dropout — Randomly disables neurons during training.

100 neurons

$$y = w_1 x_1 + w_2 x_2 + w_3 x_3 + \dots + w_n x_n$$

:

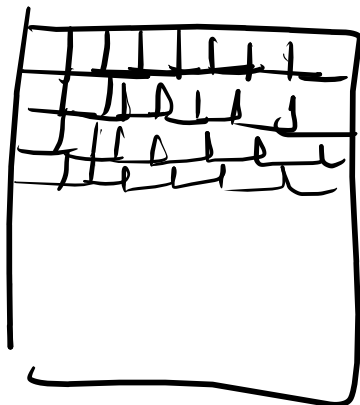
ReLU



②

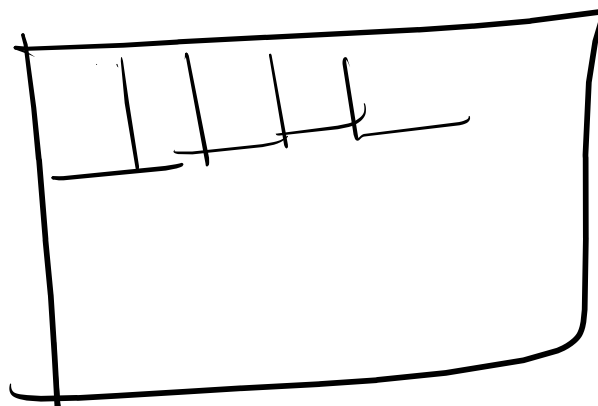
VGGNet (2014) → visual geometry group

AlexNet



slow

3x3 filters



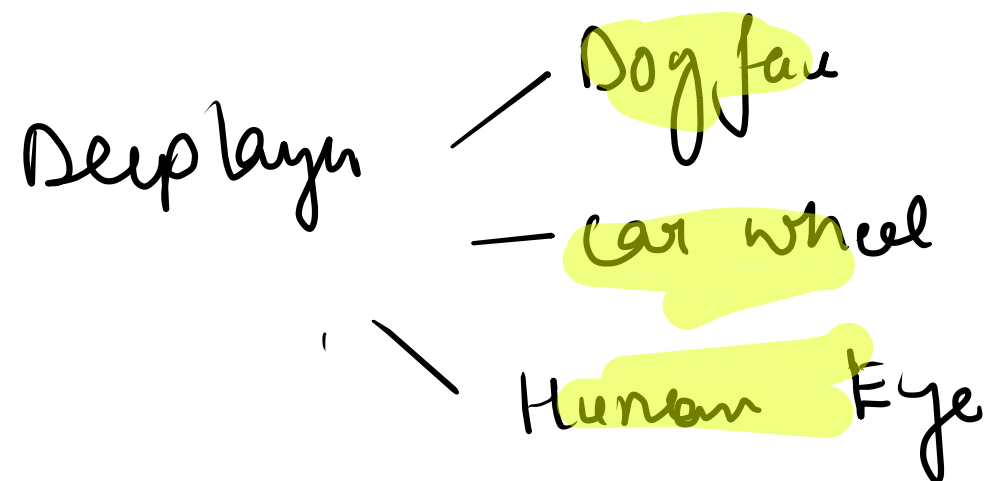
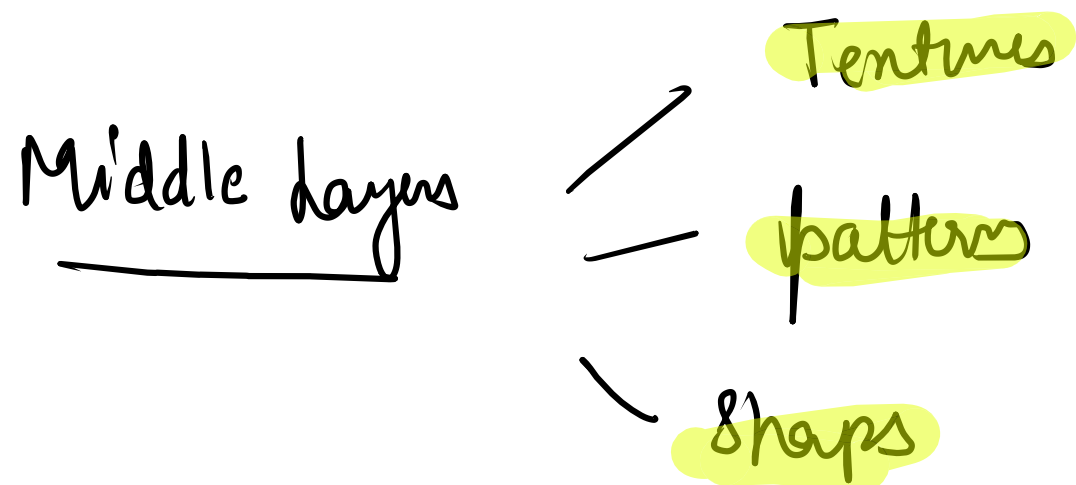
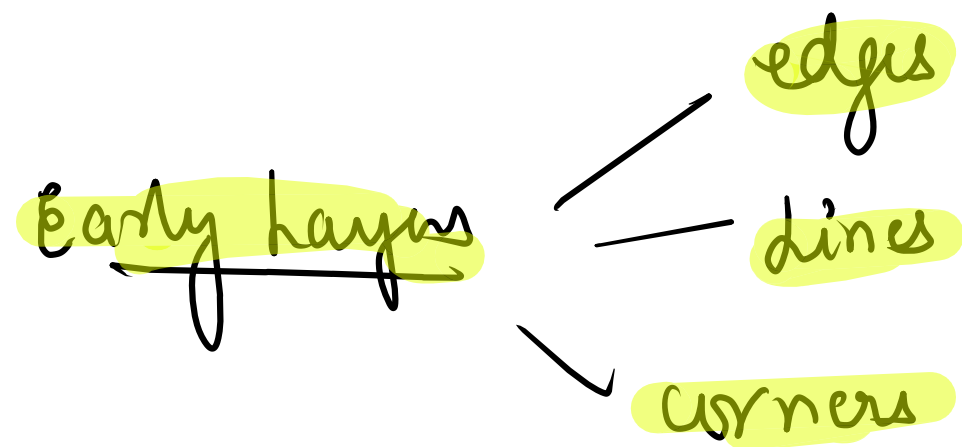
Less computation
more linearity
Better feature
interactions

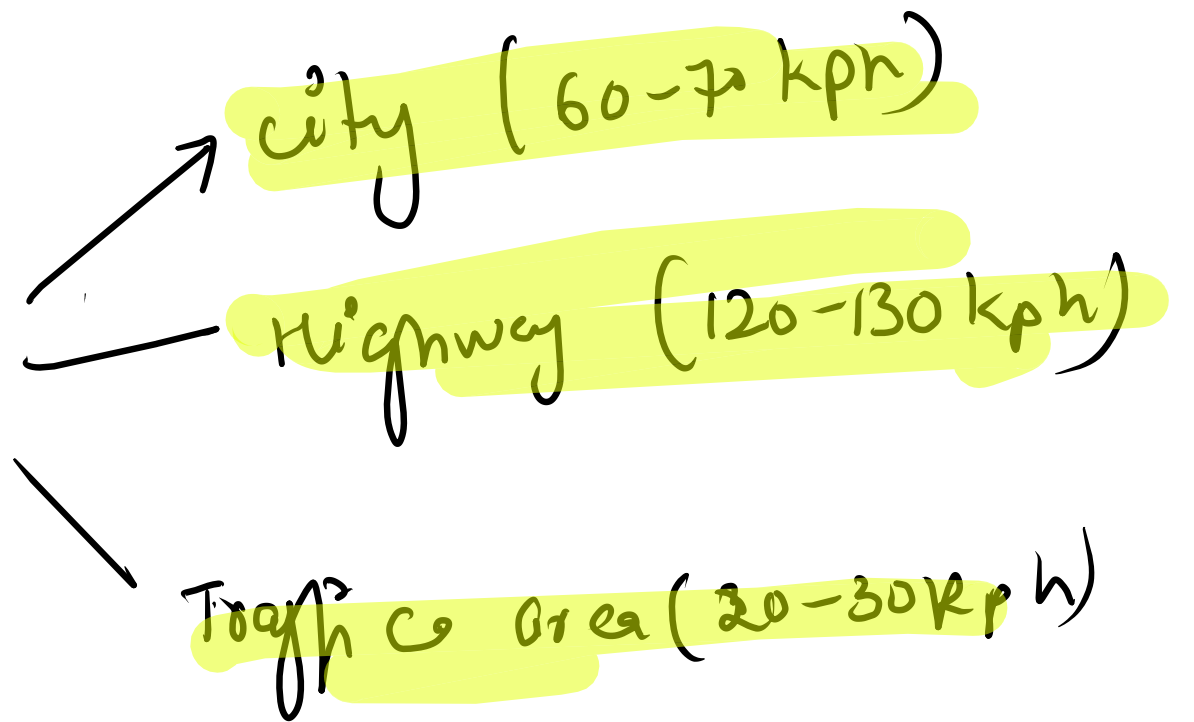
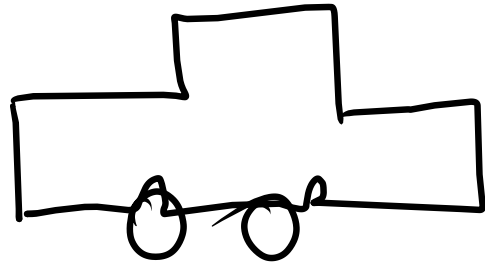
VGG architecture

Layer

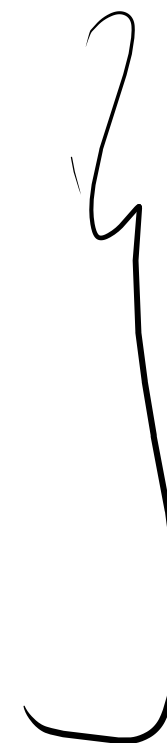
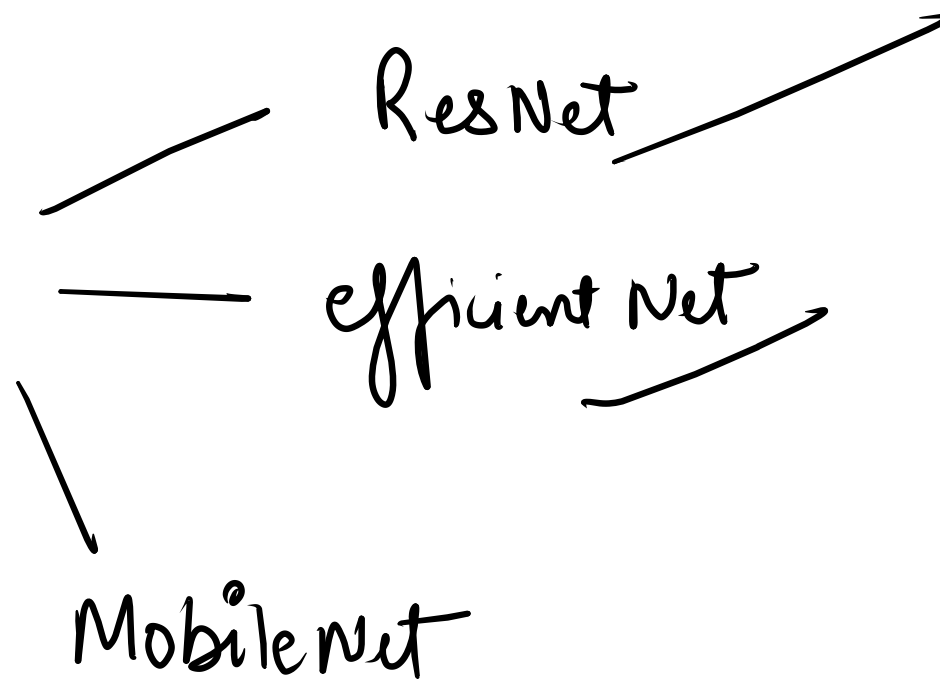
1
1

VGG feature extraction





Modern view



latest
CNN
models

Deep learning →

AlexNet (2012)



VGGNet (2014)



Resnet → efficient net

Suppose you want to build

Cat vs Dog classifier

500 Images

CNN

10,000 + Images

Millions of Images

Transfer Learning
Concept

ChatGPT

10 billion²
Images

Transfer Learning → use a model already trained
on ImageNet

These models have already learned

edges

textures

corners

shapes

textures

objects

etc etc

from millions of Images

§ # Freeze and unfreezing layers

every CNN layer has:

weight

Bias

Freeze layer → Do not update weights.

layer.trainable = false

unfreeze layer → update weights